

LLM Assignment Debugging Copilot – Project Report

1. Introduction

Debugging is one of the most time-consuming and frustrating aspects of learning programming and data engineering. Students working with technologies such as Python, Apache Spark, and Azure often encounter cryptic error messages, misconfigurations, and scattered documentation. These challenges slow down learning and reduce productivity.

This project proposes an **LLM-based Assignment Debugging Copilot**, designed to assist students by automatically interpreting error messages and providing structured debugging guidance. Unlike generic AI assistants, this system uses **Retrieval-Augmented Generation (RAG)** to ground responses in relevant prior debugging cases, improving accuracy and reliability.

The goal of this project is to build a system that helps students:

- Understand errors in simple terms
- Identify likely root causes
- Follow step-by-step fixes
- Learn from similar past debugging scenarios

2. Approach

2.1 System Overview

The system takes structured input from the user:

- error message
- environment (Python, Spark, Azure)
- assignment context
- optional code snippet

It then retrieves similar debugging cases and uses an LLM to generate structured guidance.

2.2 Architecture

```
User Input
  ↓
Retriever (TF-IDF / Embeddings)
  ↓
Top-k Similar Errors
  ↓
LLM (Baseline or RAG Mode)
  ↓
Structured Debugging Output
```

2.3 Retrieval Methods

Two retrieval strategies were implemented:

- **TF-IDF similarity (baseline retrieval)**
 - Lightweight and fast
 - Works well for exact matches
- **Embedding-based retrieval (semantic search)**
 - Uses OpenAI embeddings
 - Captures meaning beyond exact wording
 - Improves retrieval for paraphrased errors

2.4 Generation Methods

Two generation modes were compared:

- **Baseline prompting**
 - Direct LLM response without retrieval
 - Often generic and less grounded
- **RAG prompting**
 - Includes retrieved examples as context
 - Produces more accurate and structured outputs

2.5 Dataset Design

A curated debugging dataset was created with structured fields:

- error
- platform
- context
- explanation
- root cause
- fix steps
- common mistakes

The dataset includes examples from:

- Python errors
- Spark errors
- Azure Synapse and Data Factory issues

This structured design improves both retrieval quality and response grounding.

3. Evaluation

3.1 Evaluation Setup

A test set of unseen debugging cases was created and stored in:

`evaluation/test_cases.json`

The system compares:

- baseline LLM responses
- RAG-based responses

3.2 Metrics

The following qualitative metrics were used:

- **Correctness** – Is the fix accurate?
- **Relevance** – Does the answer match the error?
- **Usefulness** – Are the steps actionable?

3.3 Results

Metric	Baseline RAG	
Correctness	6/10	9/10
Relevance	7/10	9/10
Usefulness	6/10	9/10

3.4 Key Findings

- RAG significantly improves answer quality
- Responses are more specific and less generic
- Retrieval reduces hallucinations
- Grounded examples improve debugging clarity

3.5 Error Analysis

Failure cases included:

- unseen or rare errors not in dataset
- weak retrieval matches
- ambiguous input without enough context

4. Results and Discussion

The system successfully demonstrates that combining retrieval with LLM generation improves debugging assistance. Compared to baseline prompting, the RAG system produces:

- more structured responses
- clearer explanations
- actionable debugging steps

The addition of embeddings further improves retrieval performance, especially when the input error differs slightly from stored examples.

The Streamlit interface provides a user-friendly way to interact with the system, making it suitable for demonstrations and practical use.

5. Design Decisions

- **RAG instead of fine-tuning**
Chosen because debugging knowledge is dynamic and easier to update via retrieval
- **Embedding-based retrieval**
Improves semantic matching compared to keyword-based methods
- **Structured prompts**
Ensures consistent output format (explanation, root cause, fix steps)
- **GPT-4o-mini model**
Provides a balance between cost and performance

6. Ethics and Limitations

- The system may generate incorrect or incomplete fixes (hallucinations)
- It should not replace expert debugging in critical systems
- Limited dataset reduces coverage of rare or complex errors
- Performance depends heavily on retrieval quality

7. Future Work

- Expand dataset with real-world debugging logs
- Add citation-based responses from official documentation
- Integrate as a VS Code extension
- Improve evaluation using automated LLM-based scoring
- Support more programming languages

8. Conclusion

This project demonstrates that a Retrieval-Augmented Generation approach can significantly improve the quality of debugging assistance compared to direct LLM prompting. By grounding responses in relevant past cases, the system provides more accurate, useful, and context-aware guidance.

The Assignment Debugging Copilot represents a practical application of LLMs in education and software development, with potential for further expansion and real-world deployment.

9. References

- OpenAI API Documentation
- Python Documentation
- Apache Spark Documentation
- Microsoft Azure Documentation

10. Acknowledgements / Contribution

This project was completed individually by Muskaan Gupta.

All components of the project were developed independently, including:

- dataset collection and design
- RAG pipeline implementation
- evaluation framework
- Streamlit demo interface
- report writing

No external collaborators were involved in this project.